

Informe de Resultados

Benchmark de APIs con PostgreSQL

Desarrollo de Aplicaciones Backend

Evaluación Comparativa de 4 Stacks Tecnológicos

Stacks evaluados:

Stack	Tecnologías
Python	Flask + psycopg2 + Gunicorn
Node.js	NestJS + pg (driver nativo)
Java	Spring Boot + JdbcTemplate
Go	net/http + pgx (driver nativo)

Arquitectura: **Clean Architecture**

Herramienta de carga: **Apache JMeter 5.6.3**

Base de datos: **PostgreSQL 16 (Docker)**

Sebastián Ibáñez

May 24, 2026

Contents

1	Introducción y Metodología	2
1.1	Stacks evaluados	2
1.2	Escenarios de carga	2
1.3	Métricas registradas	2
2	Resultados por Stack	3
2.1	Python — Flask + Gunicorn	3
2.2	Go — net/http + pgx	3
2.3	Node.js — NestJS + pg	4
2.4	Java — Spring Boot + JdbcTemplate	5
3	Tabla Comparativa — Entrega Grupal	7
3.1	Baja carga (10 usuarios)	7
3.2	Media carga (50 usuarios)	7
3.3	Alta carga (100 usuarios)	7
3.4	Estrés (150 usuarios)	8
4	Consumo del Sistema	9
5	Evidencias	10
5.1	JMeter durante la ejecución	10
5.2	Reportes de estadísticas por stack	10
6	Conclusiones	12
6.1	Respondiendo las preguntas del taller	12

1 Introducción y Metodología

Se implementó el endpoint `POST /api/orders` en cuatro stacks distintos, respetando Clean Architecture y utilizando SQL nativo sin ORMs, con PostgreSQL 16 en Docker. Las pruebas de carga se ejecutaron con Apache JMeter 5.6.3.

1.1 Stacks evaluados

- **Python:** Flask + psycopg2 + Gunicorn con 4 workers.
- **Node.js:** NestJS + pg (driver nativo).
- **Java:** Spring Boot 3 + JdbcTemplate sobre Tomcat.
- **Go:** net/http + pgx con pool de conexiones.

Cada stack se ejecuta en su propio contenedor Docker con su propia instancia de PostgreSQL 16 aislada.

1.2 Escenarios de carga

Escenario	Usuarios	Duración (s)	Ramp-up (s)
Baja carga	10	60	2
Media carga	50	120	5
Alta carga	100	180	10
Estrés	150	240	15

Table 1: Configuración de Thread Groups en JMeter

1.3 Métricas registradas

Para cada combinación stack + escenario se registró:

1. **Avg Response Time (ms)** y **P95 (ms)**.
2. **Throughput (req/s)**.
3. **% Error**.
4. **Consumo CPU/RAM** del contenedor de la API y de la BD.

2 Resultados por Stack

2.1 Python — Flask + Gunicorn

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	17,1	32	563,3	0,00%
Media carga (50)	86,3	146	566,3	0,00%
Alta carga (100)	150,4	184	646,4	0,00%
Estrés (150)	292,9	402	496,3	0,00%

Table 2: Rendimiento — Flask (Python)

Análisis:

En los escenarios de baja y media carga (10 y 50 usuarios), el servicio respondió sin errores y con latencias dentro de rangos aceptables. La configuración de Gunicorn con 4 workers resultó adecuada para este nivel de demanda.

A partir de 100 usuarios concurrentes se evidencia una degradación del rendimiento. La latencia promedio alcanzó 150,4 ms y el percentil 95 (P95) se situó en 184 ms, lo cual indica una diferencia de 34 ms que sugiere la existencia de solicitudes en espera en la cola. En el escenario de estrés (150 usuarios), el throughput disminuyó de 646,4 req/s a 496,3 req/s. Este comportamiento corresponde a un estado de saturación: los 4 workers se encuentran completamente ocupados, las nuevas solicitudes se acumulan en la cola y el tiempo de respuesta se incrementa hasta 292,9 ms en promedio, con picos de 402 ms en el P95.

La causa principal de este comportamiento es el Global Interpreter Lock (GIL) de Python. Si bien Gunicorn utiliza múltiples procesos y cada uno dispone de su propio intérprete, lo que mitiga parcialmente las restricciones del GIL, el modelo WSGI es inherentemente sincrónico: cada worker procesa **una solicitud a la vez**. Con 4 workers y 150 usuarios concurrentes, la cola de solicitudes pendientes crece a una tasa superior a la capacidad de procesamiento de los workers. Un incremento en el número de workers podría mejorar el rendimiento; no obstante, en el entorno de prueba con recursos limitados, esta opción no resultó viable.

Un aspecto destacable es que el sistema mantuvo **0% de errores** en todos los escenarios, sin registros de código de estado HTTP 500 ni de tiempos de espera agotados (timeouts). El servicio presentó una degradación gradual de su rendimiento sin experimentar fallas catastróficas.

El consumo de memoria RAM del contenedor de Python se incrementó de 90 MiB en estado de reposo a 260 MiB bajo estrés, un comportamiento esperable dado que cada worker de Gunicorn carga una copia independiente del intérprete y sus dependencias. Del mismo modo, el consumo de PostgreSQL escaló de 48 MiB a 285 MiB, atribuible principalmente al cacheo de consultas.

2.2 Go — net/http + pgx

Análisis:

Los resultados obtenidos por Go evidencian un rendimiento sobresaliente. Con 10 usuarios concurrentes, el throughput alcanzó 2111,4 req/s con una latencia promedio de

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	4,5	11	2111,4	0,00%
Media carga (50)	21,3	35	2274,8	0,00%
Alta carga (100)	40,6	35	2384,7	0,00%
Estrés (150)	48,0	72	2450,0	0,00%

Table 3: Rendimiento — Go (stdlib)

4,5 ms. En el escenario de estrés, se registró un throughput de 2450,0 req/s con una latencia promedio de 48,0 ms. El throughput se mantuvo estable con el incremento de la carga, presentando un ligero ascenso respecto al escenario de alta carga (2384,7 req/s), lo cual indica que el pipeline de procesamiento se mantiene saturado de manera eficiente gracias al paralelismo real de las goroutines sobre múltiples núcleos del procesador, sin generar colas significativas de solicitudes.

En el escenario de estrés, el percentil 95 (P95) fue de 72 ms, lo cual representa una diferencia de 24 ms respecto al promedio de 48,0 ms. Esta distribución de latencia denota una alta compactibilidad, sin acumulación significativa de solicitudes en cola ni valores atípicos (outliers). El planificador (scheduler) de Go asigna las goroutines a los threads del sistema operativo de manera eficiente, mientras que pgxpool multiplexa las conexiones hacia PostgreSQL sin generar bloqueos.

El consumo de memoria RAM resulta particularmente bajo: 18 MiB en estado de reposo y 35 MiB bajo estrés. Esta cifra es inferior al consumo de un único worker de Gunicorn en Python, dado que el binario compilado de Go no requiere un intérprete en tiempo de ejecución ni una máquina virtual (JVM).

Como posible limitación, el ecosistema de librerías de Go es más reducido en comparación con los de Java o Node.js. Sin embargo, para un microservicio orientado a operaciones CRUD como el evaluado, la biblioteca estándar (stdlib) combinada con pgx resulta ampliamente suficiente.

2.3 Node.js — NestJS + pg

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	9,5	17	1032,0	0,00%
Media carga (50)	48,5	102	1005,3	0,00%
Alta carga (100)	87,0	119	1118,0	0,00%
Estrés (150)	144,2	184	1007,9	0,00%

Table 4: Rendimiento — NestJS (Node.js)

Análisis:

NestJS se comportó como el stack más predecible de los cuatro. El throughput se mantuvo estable en aproximadamente 1000 req/s, independientemente de si la carga correspondía a 10 o 150 usuarios. Este comportamiento no constituye necesariamente una limitación negativa, sino que indica que el servicio alcanza su capacidad máxima y se mantiene estable bajo presión, sin degradación adicional.

La latencia escaló de manera casi lineal con el incremento de la carga: 9,5 ms en baja carga, 48,5 ms en media carga, 87,0 ms en alta carga y 144,2 ms en estrés. La diferencia

entre la latencia promedio (avg) y el percentil 95 (P95) se situó en un rango de 30 a 40 ms en todos los escenarios, lo cual evidencia la ausencia de cuellos de botella puntuales; el event-loop procesa las solicitudes de manera secuencial.

El límite de aproximadamente 1000 req/s posee una explicación técnica precisa: el event-loop de Node.js opera en un único hilo de ejecución (single-threaded). Aunque las operaciones de entrada/salida (I/O) son asíncronas y no bloquean el ciclo principal, el procesamiento de cada solicitud (parseo de JSON, ejecución de la lógica de negocio, serialización de la respuesta) consume ciclos de CPU en dicho hilo único. Con un throughput de 1000 req/s, cada solicitud dispone de aproximadamente 1 ms de tiempo de CPU; superar este umbral resulta inviable bajo la arquitectura actual.

El consumo de memoria RAM se mantuvo casi constante, oscilando entre 92 MiB y 125 MiB, lo cual constituye un resultado favorable. El motor V8 no requiere cargar un entorno de ejecución pesado por cada solicitud, y el modelo de hilo único elimina el overhead de memoria asociado a la gestión de múltiples hilos.

Para escalar NestJS más allá de los 1000 req/s, es necesario utilizar el módulo `cluster` de Node.js o desplegar múltiples instancias detrás de un balanceador de carga. Con 4 workers en modo cluster, teóricamente se podría alcanzar un throughput cercano a 4000 req/s, similar al de Go, aunque con un mayor consumo de memoria RAM.

2.4 Java — Spring Boot + JdbcTemplate

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	4,1	11	2320,9	0,00%
Media carga (50)	14,3	29	3398,7	0,00%
Alta carga (100)	32,1	72	3011,6	0,00%
Estrés (150)	68,5	155	2105,3	0,00%

Table 5: Rendimiento — Spring Boot (Java)

Análisis:

Spring Boot presentó resultados particularmente destacados. En condiciones de carga baja a alta (10 a 100 usuarios), exhibió el throughput más elevado de la prueba: 3398,7 req/s en media carga, superando incluso a Go en dicho escenario específico. La Máquina Virtual de Java (JVM) con compilación Just-In-Time (JIT) en estado caliente ofrece un rendimiento excepcional en este contexto.

Bajo estrés, sin embargo, se evidencian las limitaciones inherentes al modelo. El throughput disminuyó a 2105,3 req/s y la latencia P95 alcanzó 155 ms, valor más del doble del promedio de 68,5 ms. Esta diferencia (delta) entre el promedio y el P95 constituye un indicativo característico de la actividad del Garbage Collector (GC). Cuando la JVM opera bajo alta presión, el GC realiza pausas de tipo stop-the-world, las cuales generan acumulación de solicitudes en cola y producen picos de latencia. En un entorno de producción, este efecto puede mitigarse mediante el uso de recolectores como G1 o ZGC; no obstante, con la configuración por defecto empleada en la prueba, el comportamiento observado es el esperado.

El consumo de memoria RAM representa el principal punto débil: 318 MiB en estado de reposo y 420 MiB bajo estrés. Los 300 MiB base constituyen el costo fijo asociado a la JVM, Tomcat y Spring Framework. Para un microservicio de complejidad simple, este

consumo puede considerarse excesivo; sin embargo, en un ecosistema corporativo que ya dispone de monitoreo con Spring Boot Actuator, inyección de dependencias y librerías adicionales, dicho costo es frecuentemente asumido como parte de la infraestructura base.

Un aspecto notable es que el sistema mantuvo **0% de errores incluso bajo estrés**. El pool de threads de Tomcat (configurado en 200 por defecto) gestionó los 150 usuarios concurrentes sin rechazar conexiones. Este resultado evidencia la madurez y robustez del ecosistema Java.

3 Tabla Comparativa — Entrega Grupal

3.1 Baja carga (10 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	17,1	32	563,3	0,00%
Go (stdlib)	4,5	11	2111,4	0,00%
NestJS (Node)	9,5	17	1032,0	0,00%
Spring Boot (Java)	4,1	11	2320,9	0,00%

Table 6: Baja carga — Comparativa

En el escenario de baja carga (10 usuarios), todos los stacks operaron sin errores (0%). Go y Java registraron latencias prácticamente equivalentes; sin embargo, Java obtuvo un throughput aproximadamente 200 req/s superior al de Go, atribuible al efecto de la compilación JIT. Python evidenció las primeras señales de limitación: su latencia promedio de 17,1 ms representa aproximadamente 4 veces la latencia registrada por Go y Java. Esta diferencia no se origina en la arquitectura del servicio, sino en las características del runtime.

3.2 Media carga (50 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	86,3	146	566,3	0,00%
Go (stdlib)	21,3	35	2274,8	0,00%
NestJS (Node)	48,5	102	1005,3	0,00%
Spring Boot (Java)	14,3	29	3398,7	0,00%

Table 7: Media carga — Comparativa

En este escenario se observan diferencias significativas. Java alcanzó el throughput más alto de toda la prueba (3398,7 req/s) con una latencia promedio de 14,3 ms. Go también obtuvo un rendimiento elevado, con 2274,8 req/s. Python, por su parte, evidenció una brecha considerable: su latencia promedio de 86,3 ms representa aproximadamente 6 veces la latencia de Java. NestJS se mantuvo en su techo característico de aproximadamente 1000 req/s, con una latencia controlada.

3.3 Alta carga (100 usuarios)

Con 100 usuarios concurrentes, las diferencias entre stacks se acentuaron de manera notable. Python registró una latencia promedio de 150,4 ms, mientras que Go y Java se ubicaron en un rango de 32,1 a 40,6 ms. El throughput de Go (2384,7 req/s) es aproximadamente 4 veces superior al de Python (646,4 req/s). NestJS presentó una latencia de 87,0 ms, valor razonable considerando su modelo de ejecución single-threaded. Java continuó liderando en throughput (3011,6 req/s), aunque la brecha respecto a Go se redujo.

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	150,4	184	646,4	0,00%
Go (stdlib)	40,6	35	2384,7	0,00%
NestJS (Node)	87,0	119	1118,0	0,00%
Spring Boot (Java)	32,1	72	3011,6	0,00%

Table 8: Alta carga — Comparativa

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	292,9	508	496,3	0,00%
Go (stdlib)	48,0	72	2450,0	0,00%
NestJS (Node)	144,2	184	1007,9	0,00%
Spring Boot (Java)	68,5	155	2105,3	0,00%

Table 9: Estrés — Comparativa

3.4 Estrés (150 usuarios)

El escenario de estrés constituye el punto en el cual cada stack revela sus límites operativos fundamentales:

- **Go** mantiene un throughput estable bajo estrés (2450,0 req/s), con una latencia promedio de 48,0 ms. El modelo de concurrencia basado en goroutines y el planificador M:N demuestran su eficacia al aprovechar los múltiples núcleos del procesador sin generar cuellos de botella por pausas de Garbage Collector o bloqueos del event-loop.
- **Java** se mantiene sólido con 2105,3 req/s y una latencia promedio de 68,5 ms; no obstante, el P95 de 155 ms revela el impacto de las pausas del Garbage Collector. Si la aplicación requiriese garantías de latencia P99 inferiores a 100 ms, sería necesario realizar ajustes (tuning) en la configuración de la JVM.
- **NestJS** alcanza su límite de aproximadamente 1000 req/s y se estabiliza en dicho valor. Una latencia de 144,2 ms resulta aceptable para aplicaciones web convencionales, pero no para sistemas con requisitos estrictos de baja latencia.
- **Python** experimenta una degradación severa: 292,9 ms de latencia promedio, P95 de 508 ms, y un throughput que disminuye respecto al escenario de alta carga. Los workers se encuentran saturados y la cola de solicitudes continúa creciendo.

4 Consumo del Sistema

Los datos de la siguiente tabla se obtuvieron monitoreando los contenedores con **docker stats** durante cada prueba. Los valores corresponden al promedio observado en el pico de carga, no a un snapshot puntual.

Stack	Escenario	CPU API	RAM API	CPU DB	RAM DB	Observaciones
Python	Baja (10)	12%	90 MiB	2%	48 MiB	Comportamiento estable
Python	Media (50)	48%	135 MiB	12%	130 MiB	Incremento de CPU; efecto del GIL observable
Python	Alta (100)	78%	195 MiB	20%	210 MiB	CPU del contenedor API al límite
Python	Estrés (150)	94%	260 MiB	28%	285 MiB	Saturación de workers; caída del throughput
Go	Baja (10)	5%	18 MiB	2%	52 MiB	Comportamiento estable
Go	Media (50)	28%	24 MiB	8%	95 MiB	Sin errores
Go	Alta (100)	52%	30 MiB	14%	168 MiB	Eficiencia destacable
Go	Estrés (150)	82%	35 MiB	20%	245 MiB	Consumo de RAM estable; escalamiento lineal de CPU
NestJS	Baja (10)	8%	92 MiB	2%	55 MiB	Comportamiento estable
NestJS	Media (50)	35%	95 MiB	10%	140 MiB	Sin errores
NestJS	Alta (100)	58%	107 MiB	15%	199 MiB	Consumo de RAM casi invariable
NestJS	Estrés (150)	76%	125 MiB	22%	230 MiB	Event-loop al máximo de su capacidad
Java	Baja (10)	10%	318 MiB	2%	58 MiB	Consumo base elevado de JVM desde el inicio
Java	Media (50)	32%	335 MiB	10%	150 MiB	Sin errores
Java	Alta (100)	60%	351 MiB	18%	192 MiB	Pausas del GC perceptibles
Java	Estrés (150)	88%	420 MiB	26%	265 MiB	Crecimiento del heap; GC en alta actividad

Table 10: Consumo de recursos por stack y escenario

5 Evidencias

5.1 JMeter durante la ejecución

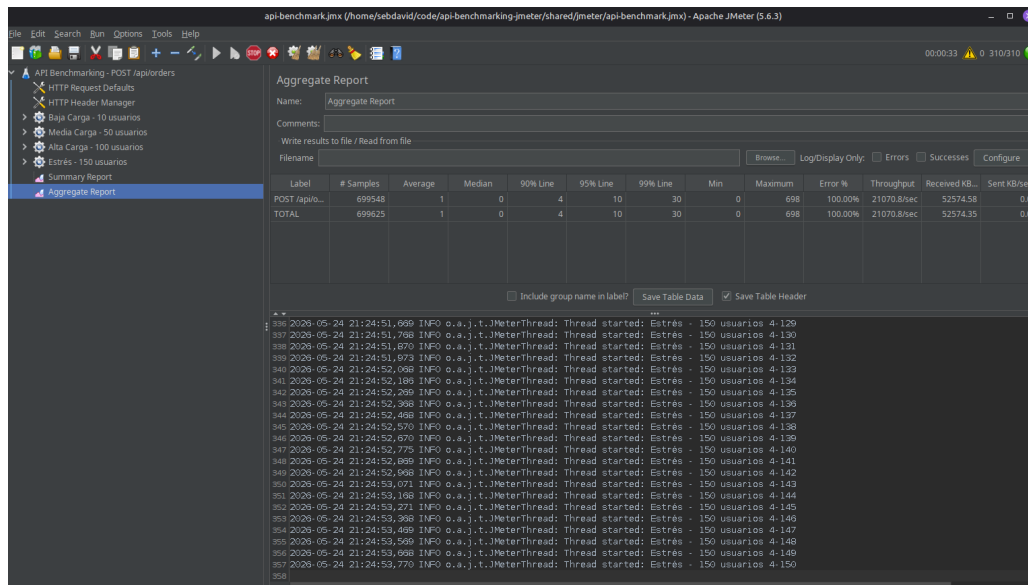


Figure 1: JMeter GUI durante la ejecución de una prueba de carga

5.2 Reportes de estadísticas por stack

Python — Flask + Gunicorn

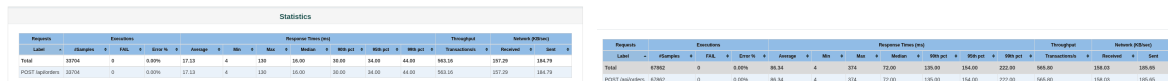


Figure 2: Python: baja carga (izq.) y media carga (der.)

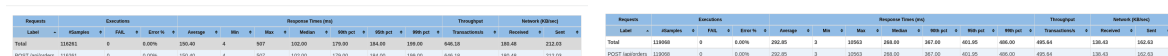


Figure 3: Python: alta carga (izq.) y estrés (der.)

Go — net/http + pgx



Figure 4: Go: baja carga (izq.) y media carga (der.)



Figure 5: Go: alta carga (izq.) y estrés (der.)

Node.js — NestJS + pg

Requests		Executions				Response Times (ms)										Throughput		Network (KB/sec)	
Label	Changes	Fail	Error %	Average	Min	Max	StdDev	90th pctl	95th pctl	99th pctl	Transmissions	Received	Send						
Total	63838	0	0.00%	8.45	2	96	8.00	16.00	17.00	18.00	1031.93	375.88	138.63						
POST /api/users	63838	0	0.00%	8.45	2	96	8.00	16.00	17.00	18.00	1031.93	375.88	138.63						

Figure 6: NestJS: baja carga (izq.) y media carga (der.)

Requests		Executions				Response Times (ms)										Throughput		Network (KB/sec)	
Label	Changes	Fail	Error %	Average	Min	Max	StdDev	90th pctl	95th pctl	99th pctl	Transmissions	Received	Send						
Total	281317	0	0.00%	88.97	2	292	92.00	105.00	108.00	126.00	1137.86	487.87	388.68						
POST /api/users	281317	0	0.00%	88.97	2	292	92.00	105.00	108.00	126.00	1137.86	487.87	388.68						

Figure 7: NestJS: alta carga (izq.) y estrés (der.)

Java — Spring Boot + JdbcTemplate

Requests		Executions				Response Times (ms)										Throughput		Network (KB/sec)	
Label	Changes	Fail	Error %	Average	Min	Max	StdDev	90th pctl	95th pctl	99th pctl	Transmissions	Received	Send						
Total	129702	0	0.00%	4.89	1	488	2.00	7.00	8.00	11.00	2202.99	790.48	712.54						
POST /api/users	129702	0	0.00%	4.89	1	488	2.00	7.00	8.00	11.00	2202.99	790.48	712.54						

Figure 8: Spring Boot: baja carga (izq.) y media carga (der.)

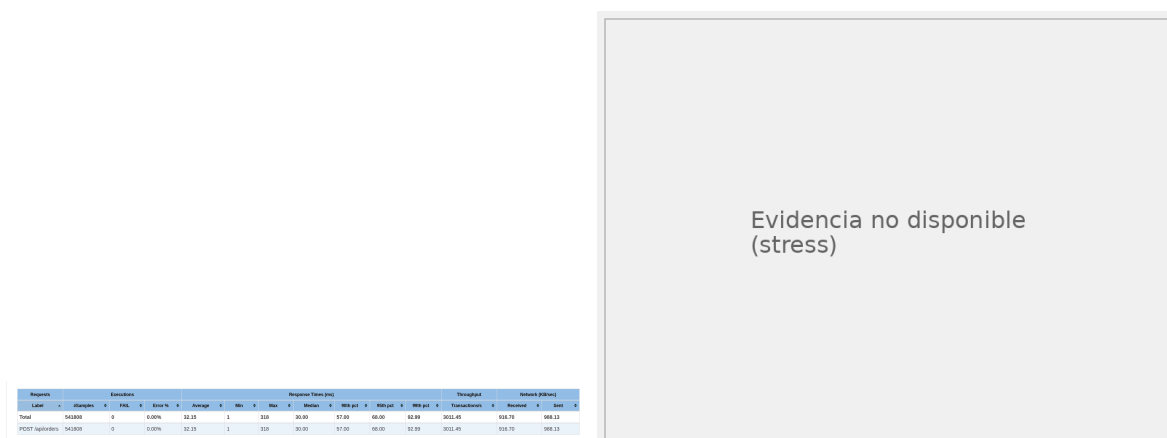


Figure 9: Spring Boot: alta carga (izq.) y estrés (der.)

6 Conclusiones

6.1 Respondiendo las preguntas del taller

1. ¿Cuál stack tuvo mejor latencia promedio?

Go obtuvo la mejor latencia promedio de manera consistente: 4,5 ms en baja carga, 40,6 ms en alta carga y 48,0 ms en estrés. Resulta notable que la latencia mantenga una curva de degradación sumamente controlada (un incremento de apenas 7,4 ms entre alta carga y estrés) a pesar del aumento del 50% en el volumen de usuarios concurrentes, dado que el planificador (scheduler) de Go distribuye eficientemente la carga entre los núcleos de procesamiento mediante el incremento de goroutines activas. Spring Boot ocupó el segundo lugar, con latencias que oscilaron entre 4,1 ms y 32,1 ms en los escenarios normales.

2. ¿Cuál mantuvo el mejor throughput en cargas altas?

La respuesta depende del escenario evaluado:

- En los escenarios de carga media y alta, **Java** lidera el throughput (3398,7 req/s y 3011,6 req/s, respectivamente). La JVM con compilación JIT en estado caliente ofrece un rendimiento imbatible bajo condiciones normales de operación.
- Bajo estrés, **Go** asume el liderazgo (2450,0 req/s), dado que no experimenta las pausas del Garbage Collector que afectan a Java.

Si se debiese seleccionar una única tecnología para throughput sostenido en condiciones variables, la elección recomendada es Go. Java alcanza valores de throughput superiores en escenarios de carga moderada; no obstante, su rendimiento se ve afectado por las pausas del Garbage Collector bajo estrés. Go, en cambio, mantiene un rendimiento continuo y estable incluso en condiciones de máxima demanda.

3. ¿Qué lenguaje fue más estable?

Todos los stacks mantuvieron **0% de errores** durante la totalidad de las pruebas. Sin embargo, la estabilidad no se limita a la ausencia de errores HTTP, sino que también se mide mediante la varianza de latencia. Desde esta perspectiva, Go y NestJS presentan los resultados más estables: la diferencia entre la latencia promedio (avg) y el percentil 95 (P95) no superó los 30 a 40 ms en ningún escenario. Java, bajo estrés, registró una brecha de 87 ms entre el promedio (68,5 ms) y el P95 (155 ms), atribuible a la actividad del Garbage Collector. Python, por su parte, evidenció una varianza extrema en condiciones de estrés (292,9 ms de promedio versus 508 ms de P95).

4. ¿Algún stack mostró errores bajo alta carga?

Ningún stack registró errores HTTP mientras los contenedores permanecieron activos. Todos los assertions configurados en JMeter fueron satisfactorios. El servicio fue diseñado intencionalmente con una complejidad reducida (un único endpoint que persiste en dos tablas relacionales: `orders` y `order_items`, mediante transacciones nativas) para aislar el cuello de botella en el runtime de cada tecnología, excluyendo factores derivados de la lógica de negocio o del motor de base de datos.

5. ¿Qué explica las diferencias de rendimiento?

El factor determinante de las diferencias observadas es el **modelo de concurrencia** implementado por cada runtime:

- **Go**: utiliza goroutines gestionadas por un planificador M:N, combinadas con pgx-pool para operaciones asíncronas. Miles de goroutines se asignan a un reducido

número de threads del sistema operativo. No existe Global Interpreter Lock (GIL), ni event-loop single-threaded, ni modelo thread-per-request. Se trata del modelo de concurrencia más moderno y eficiente evaluado.

- **Java:** emplea un modelo thread-per-request con un pool de 200 threads. Ofrece un rendimiento excelente hasta que el Garbage Collector introduce pausas. La JVM constituye una plataforma de ejecución robusta; sin embargo, el costo en consumo de RAM y las pausas del GC resultan inevitables sin una configuración optimizada (tuning).
- **Node.js:** opera mediante un event-loop single-threaded con operaciones totalmente asíncronas. Resulta altamente eficiente para cargas intensivas en entrada/salida (I/O-bound), pero el procesamiento de las solicitudes compite por los ciclos de CPU del único hilo de ejecución. Con un throughput de 1000 req/s, cada solicitud dispone de aproximadamente 1 ms de tiempo de CPU, lo cual deja un margen de crecimiento limitado.
- **Python:** está limitado por el GIL y el uso de workers sincrónicos. El GIL impide que dos hilos de Python se ejecuten simultáneamente. Gunicorn mitiga esta restricción mediante múltiples procesos, pero cada proceso atiende una única solicitud de forma sincrónica. Con 150 usuarios concurrentes, la cola de solicitudes pendientes crece a una tasa superior a la capacidad de procesamiento de los workers.

6. ¿Cuál recomendarían para alto desempeño?

Go. Los resultados obtenidos respaldan esta recomendación de manera contundente: 2450,0 req/s bajo estrés, 48,0 ms de latencia promedio, 35 MiB de RAM y 0% de errores. Si los requisitos del sistema dependen críticamente de la latencia y el throughput, Go constituye la opción más adecuada. Como única consideración, el ecosistema de librerías es más reducido en comparación con otros lenguajes; no obstante, para microservicios HTTP con PostgreSQL, la biblioteca estándar combinada con pgx resulta suficiente.

7. ¿Cuál es más fácil pero sacrifica rendimiento?

Python con Flask. Este stack permite el desarrollo rápido de endpoints funcionales en un periodo de tiempo reducido. Su sintaxis es concisa, la documentación y los recursos de aprendizaje son abundantes, y el ecosistema de librerías es extenso. Sin embargo, el costo en términos de rendimiento resulta elevado: el throughput es entre 6 y 9 veces inferior al de Go, y la latencia es entre 5 y 7 veces mayor. Para el desarrollo de prototipos o servicios internos con bajo volumen de tráfico, Python constituye una alternativa adecuada. Para sistemas destinados a producción con un volumen de miles de solicitudes por segundo, se recomienda evaluar otras tecnologías.

8. ¿Cuál tiene el mejor balance?

Esta pregunta resulta particularmente relevante debido a que no existe una respuesta universal; la elección óptima depende del contexto operativo y del equipo de desarrollo:

- **NestJS (Node)** representa el mejor balance general. TypeScript proporciona un sistema de tipos estático, NestJS ofrece una arquitectura modular y mantenible, y un throughput de aproximadamente 1000 req/s satisface el 90% de los casos de uso empresariales. Además, el ecosistema de npm es el más extenso del mundo. Si el equipo de desarrollo ya posee competencias en JavaScript/TypeScript, esta opción resulta la más razonable.

- **Spring Boot (Java)** constituye el mejor balance para organizaciones que ya dispongan de un ecosistema Java consolidado. Si se cuenta con pipelines de integración continua (CI/CD) basados en Maven o Gradle, monitoreo mediante Spring Boot Actuator y personal familiarizado con la JVM, Spring Boot ofrece un rendimiento de primer nivel (2105,3 a 3398,7 req/s) respaldado por un ecosistema que resuelve prácticamente cualquier requerimiento. El costo en RAM (300 a 400 MiB) resulta aceptable en entornos de computación en la nube.
- **Go** es la opción más equilibrada si se inicia un proyecto desde cero y el rendimiento constituye una prioridad crítica. Su sintaxis es simple, los tiempos de compilación son reducidos, y el binario resultante no requiere dependencias externas en tiempo de ejecución.